

VS

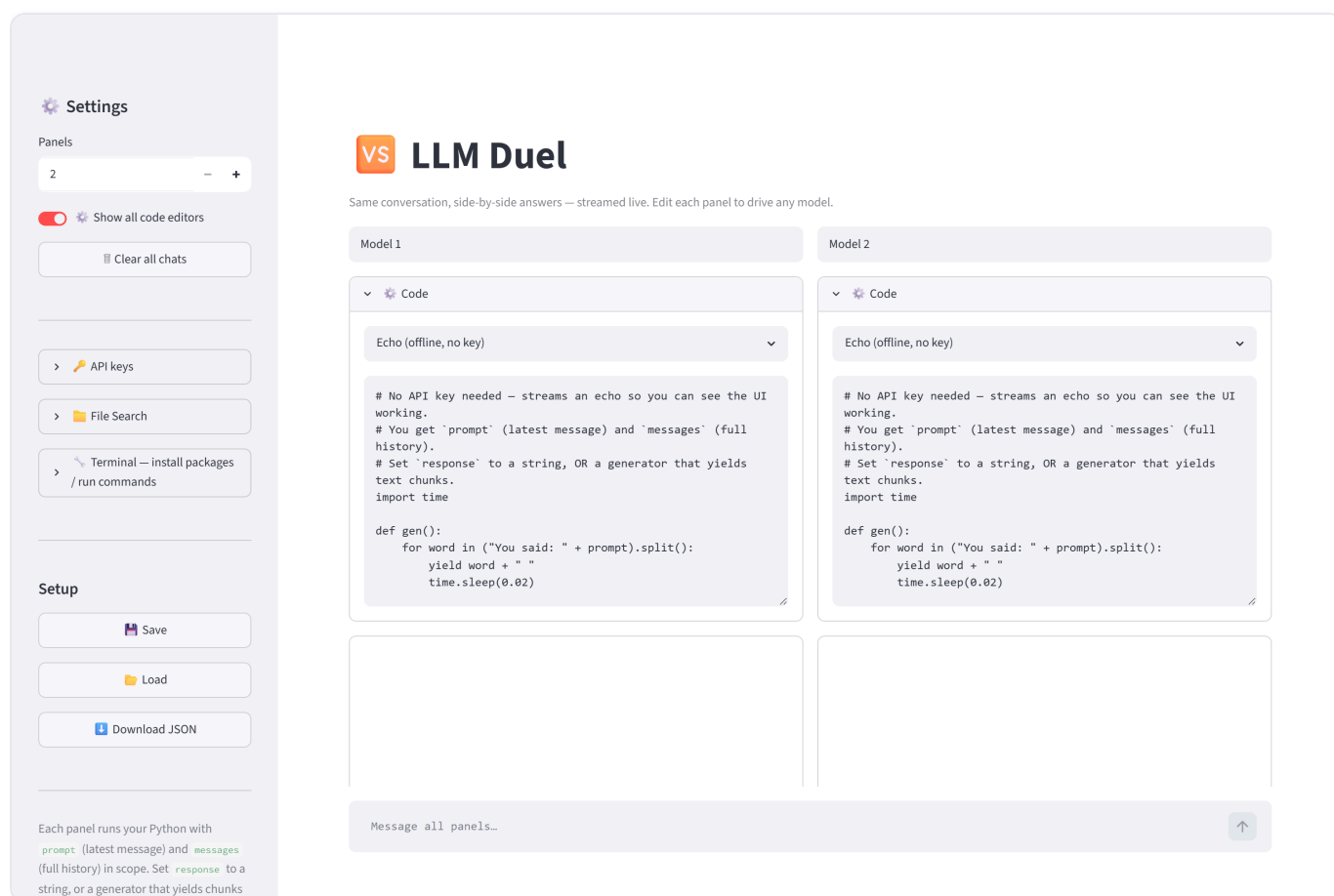
LLM Duel

User Manual — run one prompt against two LLMs, side by side.

Streamlit application · June 2026

1 What it is

LLM Duel lets you hold the **same conversation** with two (or more) language models at once and watch their answers **stream in side by side**. Each model is driven by a small block of Python you can edit in the app, so you can wire up ChatGPT, Claude, Gemini, a local model, or your own — without the app needing to know how any of them work.



The main window: settings sidebar (left), two model panels each with an editable code box, and a shared message box at the bottom.

2 Install & run

You need Python with Streamlit installed. From the project folder:

```
pip install -r requirements.txt
```

```
copy .env.example .env      # then paste in your API keys (optional)
```

Then start it in any of these ways:

- **Double-click** `run.bat` — the quickest way.
- Or run `streamlit run app.py` in a terminal.

Your browser opens at `http://localhost:8501`. It works immediately with the built-in **Echo** template — no API keys required — so you can try the interface right away.

3 The interface

Sidebar (Settings)

- **Panels** — compare 2, 3, or 4 models at once.
- **Clear all chats** — wipe every conversation and start fresh.
- **Save / Load / Download JSON** — store your panel names + code to `llm_duel_config.json` so your setup survives a restart.

Each model panel

- **Display name** — label the panel (e.g. "GPT-4o", "Claude").
-  **Code** — an editable Python box plus a template menu. It opens automatically and collapses once the conversation starts, to keep the focus on the chat.
- **Transcript** — the running conversation for that model.

Message box

Type once at the bottom; your message goes to *every* panel at the same time.

4 How it works — the contract

The clever part is a tiny, fixed contract that every panel's code follows. That single convention is what lets one app drive any model:

Direction	Name	What it is
in	<code>prompt</code>	The latest user message (a string).
in	<code>messages</code>	The full conversation so far — a list of <code>{"role", "content"}</code> dictionaries.
out	<code>response</code>	Either a plain string (shown at once), or a generator that yields text chunks (streamed live).

Pick a starting point from the template menu — **OpenAI, Anthropic, Gemini, a local model (Ollama), or Echo** — then edit it. A real example (OpenAI, streaming, multi-turn):

```
import os
from openai import OpenAI
```

```

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,          # full history, so it remembers the conversation
    stream=True,
)

def gen():
    for event in stream:
        if event.choices[0].delta.content:
            yield event.choices[0].delta.content

response = gen()                # a generator → streamed token by token

```

Settings

Panels: 2 — +

☒ Show all code editors

Clear all chats

API keys

File Search

Terminal — install packages / run commands

Setup

Save

Load

Download JSON

Each panel runs your Python with `prompt` (latest message) and `messages` (full history) in scope. Set `response` to a string, or a generator that yields chunks

vs LLM Duel

Same conversation, side-by-side answers — streamed live. Edit each panel to drive any model.

Model 1

Code

Echo (offline, no key)

Echo (offline, no key)

OpenAI / ChatGPT

Anthropic / Claude

Google / Gemini

Google / Gemini (File Search)

Local (Ollama / custom HTTP)

`time.sleep(0.02)`

Model 2

Code

Echo (offline, no key)

```
# No API key needed — streams an echo so you can see the UI working.
# You get `prompt` (latest message) and `messages` (full history).
# Set `response` to a string, OR a generator that yields text chunks.
import time

def gen():
    for word in ("You said: " + prompt).split():
        yield word + " "
        time.sleep(0.02)
```

Message all panels...

The template menu in each panel — a ready-made starting point for each provider.

5 Using it — a worked example

1. In each panel, pick a template from the **Code** menu and adjust the model name.
2. Put your API keys in a `.env` file (see section 8).
3. Type a question in the message box and press Enter.

4. Watch both answers stream in, then compare. Ask a follow-up — each model remembers its own thread.

Below, two panels were given different code (a plain echo vs. a small text analyzer) so you can see how distinct each side's answer can be. The 🕒 stamp under a reply shows how long it took.

Settings

Panels: 2

Show all code editors

Clear all chats

API keys

File Search

Terminal — install packages / run commands

Setup

Save

Load

Download JSON

Each panel runs your Python with `prompt` (latest message) and `messages` (full history) in scope. Set `response` to a string, or a generator that yields chunks

vs LLM Duel

Same conversation, side-by-side answers — streamed live. Edit each panel to drive any model.

Model 1

> Code

In one sentence, what is a transformer in machine learning?

You said: In one sentence, what is a transformer in machine learning?

🕒 0.25s

Model 2

> Code

In one sentence, what is a transformer in machine learning?

Words: 10 | Reversed: learning? machine in transformer a is what sentence, one In

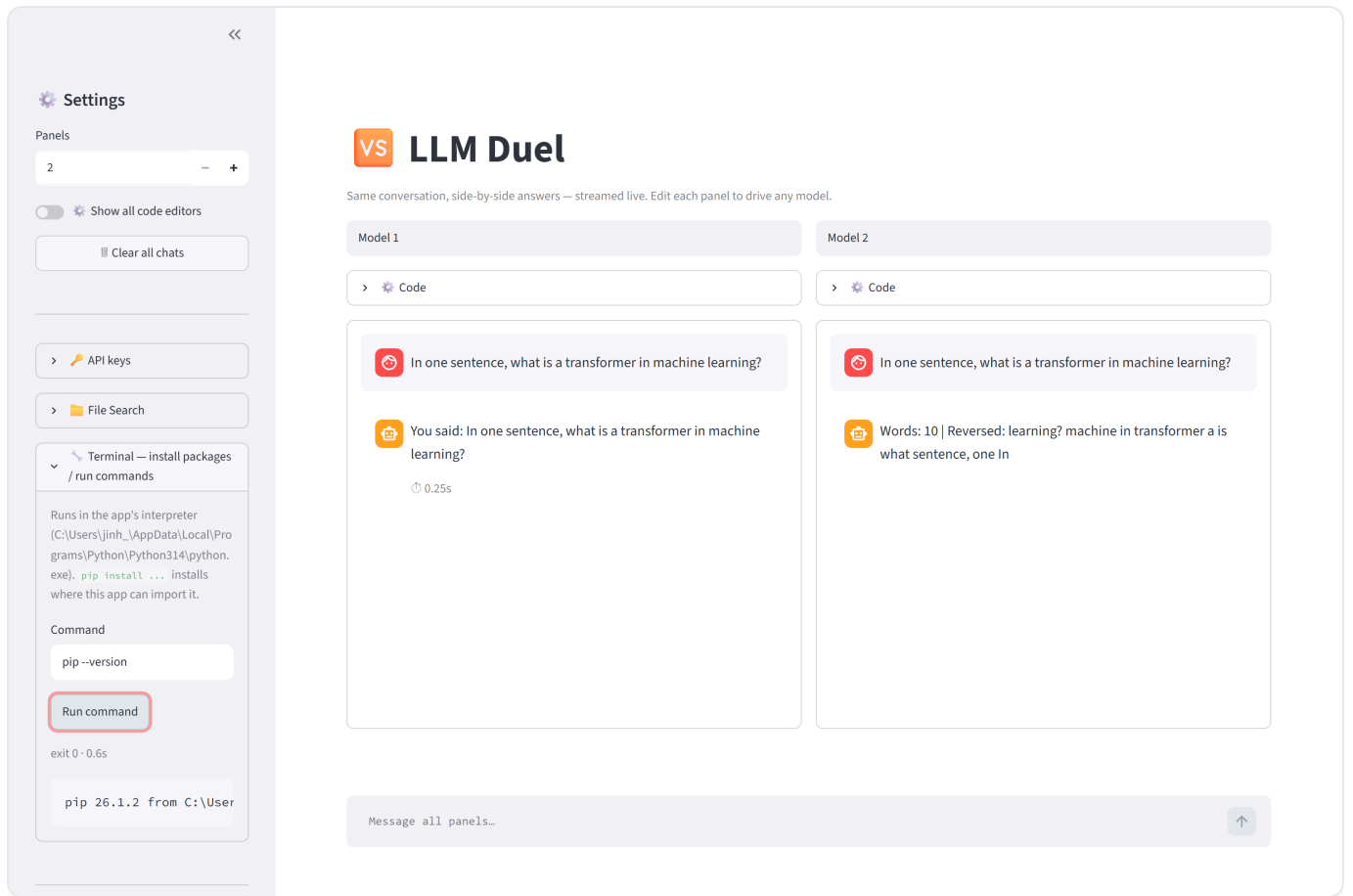
Message all panels...

Same question, two panels, two answers — streamed live and shown side by side.

6 Installing model packages (Terminal)

Different models need different Python packages. The built-in **Terminal** panel runs commands in the *same* interpreter the app uses, so anything you `pip install` there is immediately importable by your panel code.

```
pip install openai anthropic google-generativeai
```



The Terminal panel running a command and showing its output and exit code.

7 Saving setups & API keys

Use **Save** in the sidebar to write your panel names and code to `llm_duel_config.json` ; it loads automatically next time. **Download JSON** gives you a portable copy.

API keys live in a `.env` file beside `app.py` and are read by your panel code via `os.environ` :

```
OPENAI_API_KEY=sk- ...
ANTHROPIC_API_KEY=sk-ant- ...
GOOGLE_API_KEY= ...
```

8 Security & troubleshooting

Security: panels run with `exec()` and the Terminal runs shell commands — full power, by design, so any SDK works. Only paste code you trust. This is built as a local, single-user tool.

- **Browser didn't open:** go to `http://localhost:8501` manually.
- **ModuleNotFoundError in a panel:** install the package from the Terminal panel, e.g. `pip install openai`.
- **Authentication error:** check the matching key in `.env`, then restart the app.
- **"model not found":** edit the `model=" ... "` line in that panel's code.
- **One panel is slow:** that's just that model — panels stream independently and in parallel.